

Post-training

Wrap-up from previous class

Mixing, synthetic data, & more!

Where are we now?

Pre-training

Mid-training

Post-training

Goal: learn from lots of data to get general linguistic and world knowledge

Objective: language modeling

Data: raw text

Goal: Make the model usable

Objective: ??

Data: ??

Today

1. Instruction fine-tuning, or supervised fine-tuning (**SFT**)
2. Reinforcement learning from human preferences (**RLHF**), Proximal Policy Optimization (**PPO**)
3. Direct Preference Optimization (**DPO**)
4. What's happening more recently (e.g. **RLAIF**)?

Instruction Fine-tuning

Language modeling

What we have now:

Prompt:

The University of Wisconsin-Madison is located in

Model:

Madison, Wisconsin.

Language modeling: in-context learning

What we have now:

Prompt:

The dog chased a squirrel at the park. = 这只狗在公园里追一只松鼠。

I was late for class. = 我上课迟到了。

The hippopotamus ate my homework. =

Model:

河马吃了我的作业。

Language modeling $\neq$ assisting users

What we actually want:

Prompt:

Translate this to Chinese: “The hippopotamus ate my homework.”

Model:

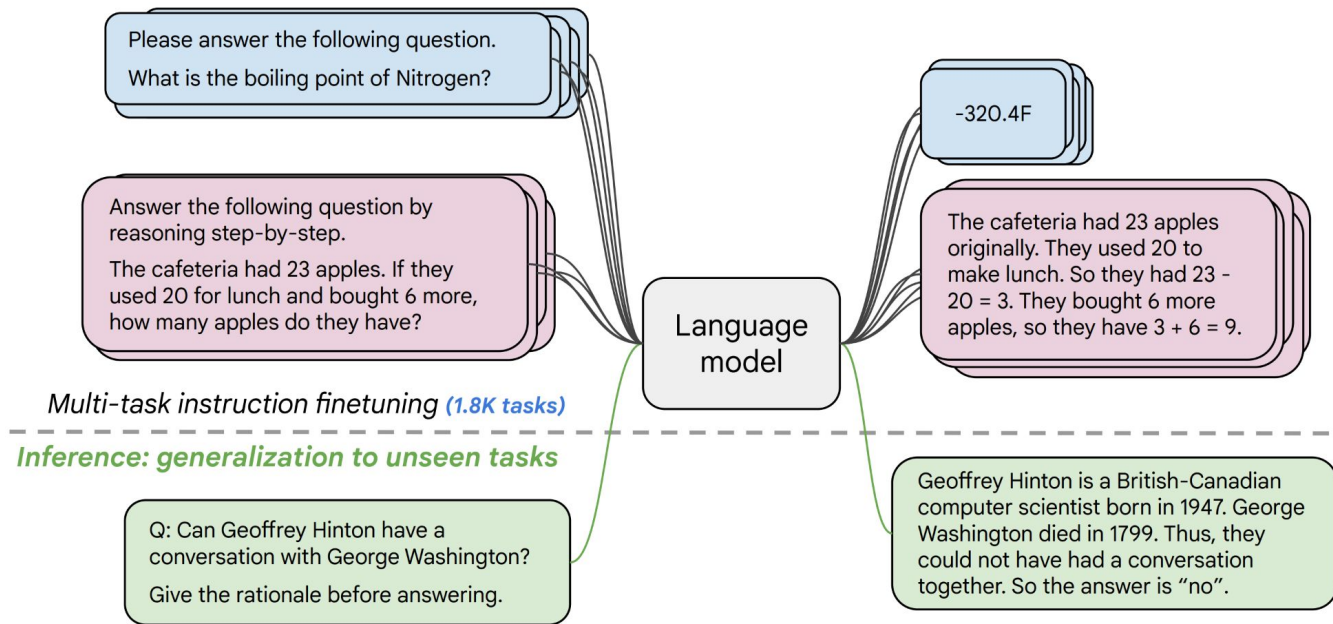
河马吃了我的作业。

We want to **align** our models with user intent.

We want to directly ask for what we want, with fine-grained control.

Instruction finetuning

Collect examples of (instruction, output) pairs across many tasks and finetune an LM



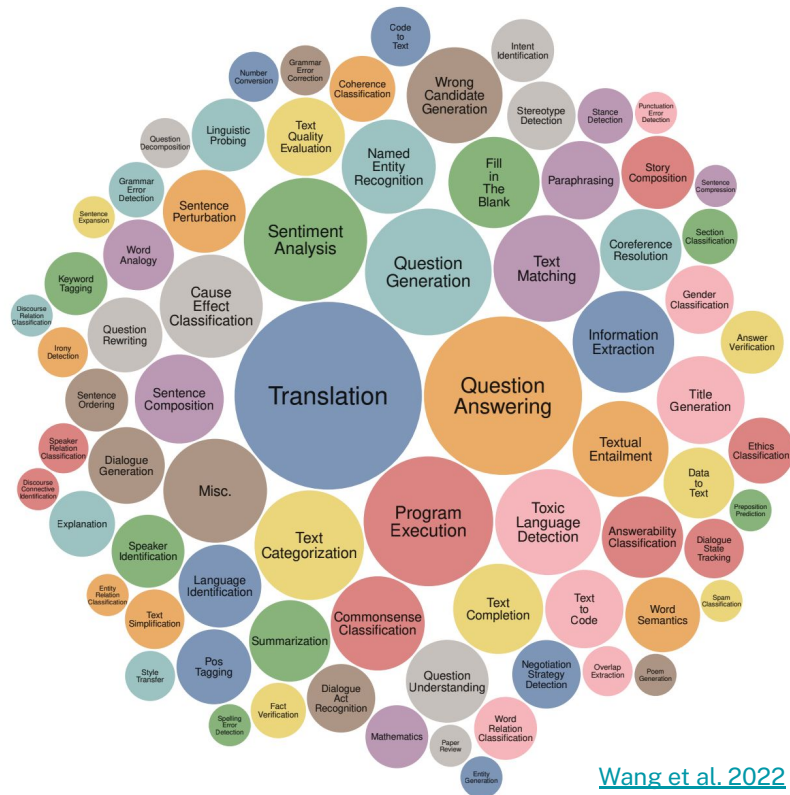
Instruction finetuning, w/ pretraining flavor

Different from *task-specific fine-tuning* (e.g. BERT in prev lecture)

The goal is to produce a **general-purpose model** that can solve many tasks! We want a large and diverse finetuning dataset.

Example:

Super-NaturalInstructions has 1.6K tasks, 3M+ examples



Wang et al. 2022

A shift to tasks that reflect real-world use

“Classic” NLP tasks

Named entity recognition. Is this word a named location, person, organization, etc?

Entailment. Does this sentence entail or contradict this other one?

Word sense disambiguation. Does the use of this word share the same or different meanings?

Coreference resolution. Can you link references to the same people or entities together?

Instruction datasets’ tasks

“Given an abstract, generate a keyword (a noun phrase) that best describes the focus or contribution of the paper.”

“Given a document, generate a short title of the document.”

“Convert a data table of restaurant descriptions into fluent natural-sounding English sentences.”

“You will be shown an incorrect English sentence. You need to generate a corrected form of the input sentence.”

Instruction tuning data formats

Without chain-of-thought

Instruction
without
exemplars

Answer the following
yes/no question.

Can you write a whole
Haiku in a single tweet?

→ yes

With chain-of-thought

Answer the following yes/no question
by reasoning step-by-step.

Can you write a whole Haiku in a
single tweet?

→

A haiku is a Japanese
three-line poem.
That is short enough
to fit in 280
characters. The
answer is yes.

Instruction
with exemplars

Q: Answer the following
yes/no question.
Could a dandelion suffer
from hepatitis?

A: no

Q: Answer the following
yes/no question.
Can you write a whole Haiku
in a single tweet?
A:

→ yes

Q: Answer the following yes/no question by
reasoning step-by-step.
Could a dandelion suffer from hepatitis?
A: Hepatitis only affects organisms with livers.
Dandelions don't have a liver. The answer is no.

Q: Answer the following yes/no question by
reasoning step-by-step.
Can you write a whole Haiku in a single tweet?
A:

→

A haiku is a Japanese
three-line poem.
That is short enough
to fit in 280
characters. The
answer is yes.

Performance on unseen/held-out tasks

Task example

Q: In the following sentences, explain the antecedent of the pronoun (which thing the pronoun refers to), or state that it is ambiguous.

Sentence: The reporter and the chef will discuss their favorite dishes.

Options:

(A) They will discuss the reporter's favorite dishes

(B) They will discuss the chef's favorite dishes

(C) Ambiguous

A: Let's think step by step.

540B base LM

The reporter and the chef will discuss their favorite dishes.

The reporter and the chef will discuss the reporter's favorite dishes.

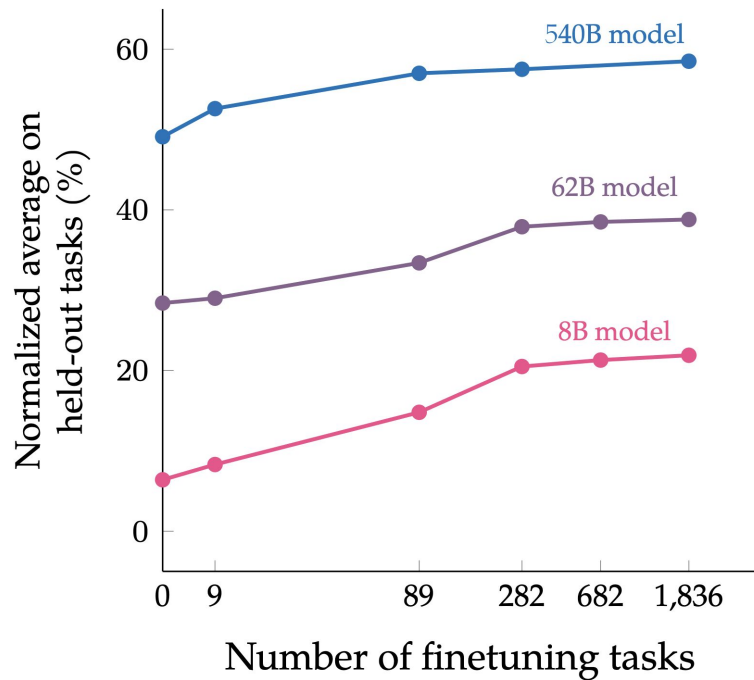
The reporter and the chef will discuss the chef's favorite dishes.

The reporter and the chef will discuss the reporter's and the chef's favorite dishes.

same LM, instruction-tuned

The reporter and the chef will discuss their favorite dishes does not indicate whose favorite dishes they will discuss. So, the answer is (C).

Performance on unseen/held-out tasks



From [Chung et al. 2022](#)

Less is more for alignment (LIMA)

You don't need many samples to
instruction tune

Fine-tuned LLaMa 65B on 1000
manually-selected examples 🚀

Source	#Examples
Training	
Stack Exchange (STEM)	200
Stack Exchange (Other)	200
wikiHow	200
Pushshift r/WritingPrompts	150
Natural Instructions	50
Paper Authors (Group A)	200
Dev	
Paper Authors (Group A)	50
Test	
Pushshift r/AskReddit	70
Paper Authors (Group B)	230

[Zhou et al., 2023](#)

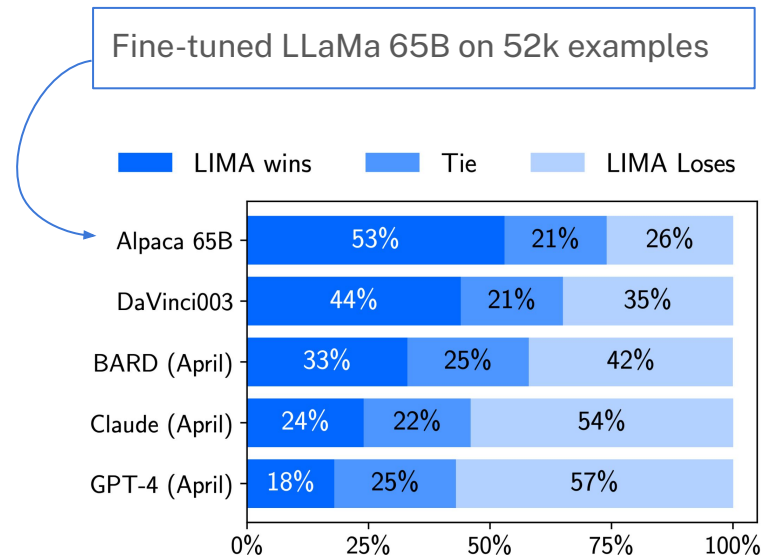
Less is more for alignment (LIMA)

You don't need many samples to instruction tune

Fine-tuned LLaMa 65B on 1000 manually-selected examples 🚀

Superficial Alignment Hypothesis

Knowledge is learned during pre-training; instruction tuning teaches models which subdistribution of formats to use.



[Zhou et al., 2023](#)

Self-Instruct

Many instruction datasets rely heavily on human-written data!

Key idea: synthetic data! Let's generate instructions, inputs, and outputs from GPT-3.

Instruction: Given an address and city, come up with the zip code.

Input:

Address: 123 Main Street, City: San Francisco

Output: 94105



Instruction: I am looking for a job and I need to fill out an application form. Can you please help me complete it?

Input:

Application Form:

Name: _____ Age: _____ Sex: _____

Phone Number: _____ Email Address: _____

Education: _____ ...

Output:

Name: John Doe Age: 25 Sex: Male

Phone Number: ...



Instruction: How to write a code for converting degrees fahrenheit to celsius.

Input: Null

Output:

```
def convert_fahrenheit_to_celsius(fahr):  
    celsius = (fahr - 32) * 5 / 9  
    return celsius
```



Self-Instruct

Many instruction datasets rely heavily on human-written data!

Key idea: synthetic data! Let's generate instructions, inputs, and outputs from GPT-3.

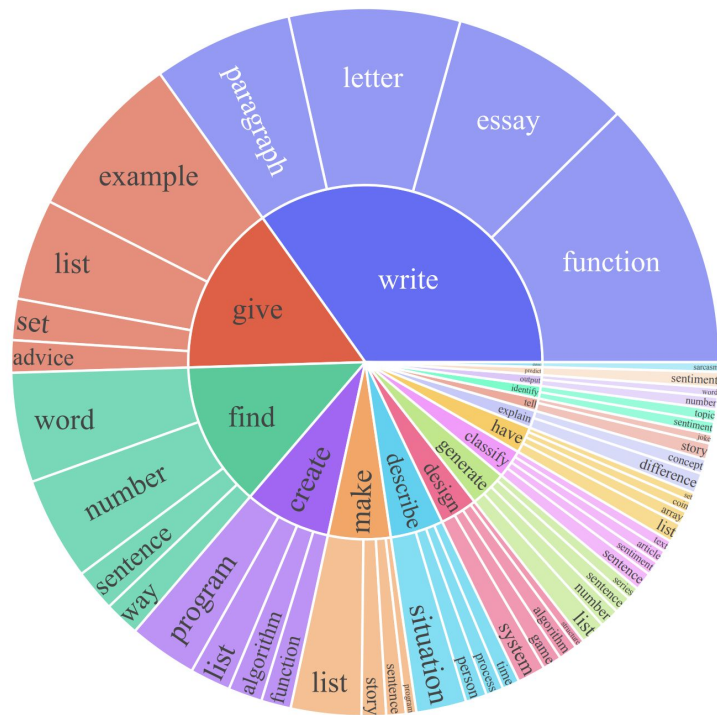
```
Come up with a series of tasks:  
Task 1: {instruction for existing task 1}  
Task 2: {instruction for existing task 2}  
Task 3: {instruction for existing task 3}  
Task 4: {instruction for existing task 4}  
Task 5: {instruction for existing task 5}  
Task 6: {instruction for existing task 6}  
Task 7: {instruction for existing task 7}  
Task 8: {instruction for existing task 8}  
Task 9:
```

[Wang et al., 2022](#)

Self-Instruct

Many instruction datasets rely heavily on human-written data!

Key idea: synthetic data! Let's generate instructions, inputs, and outputs from GPT-3.



Self-Instruct

Evaluated on expert-written instructions for novel tasks

Boosts GPT-3 performance by a lot

Matches OpenAI's instruction-tuned LM

Improves performance even when you have human-labeled instruction data

Model	# Params	ROUGE-L
Vanilla LMs		
T5-LM	11B	25.7
GPT3	175B	6.8
Instruction-tuned w/o SUPERNI		
T0	11B	33.1
GPT3 + T0 Training	175B	37.9
GPT3 _{SELF-INST} (Ours)	175B	39.9
InstructGPT ₀₀₁	175B	40.8
Instruction-tuned w/ SUPERNI		
Tk-INSTRUCT	11B	46.0
GPT3 + SUPERNI Training	175B	49.5
GPT3 _{SELF-INST} + SUPERNI Training (Ours)	175B	51.6

Self-Instruct

Evaluated on expert-written instructions for novel tasks

Stanford
Alpaca



LLaMA 7B, fine-tuned on 52K
instruction-following examples
generated by GPT-3.5

Model		# Params	ROUGE-L
Vanilla LMs			
T5-LM	11B	25.7	
GPT3	175B	6.8	
Instruction-tuned w/o SUPERNI			
① T0	11B	33.1	
GPT3 + T0 Training	175B	37.9	
② GPT3 _{SELF-INST} (Ours)	175B	39.9	
InstructGPT ₀₀₁	175B	40.8	
Instruction-tuned w/ SUPERNI			
Tk-INSTRUCT	11B	46.0	
③ GPT3 + SUPERNI Training	175B	49.5	
GPT3 _{SELF-INST} + SUPERNI Training (Ours)	175B	51.6	

Many open SFT datasets!



Different instruction-tuning datasets can help different held-out tasks, and mixtures of human + synthetic perform best on average.

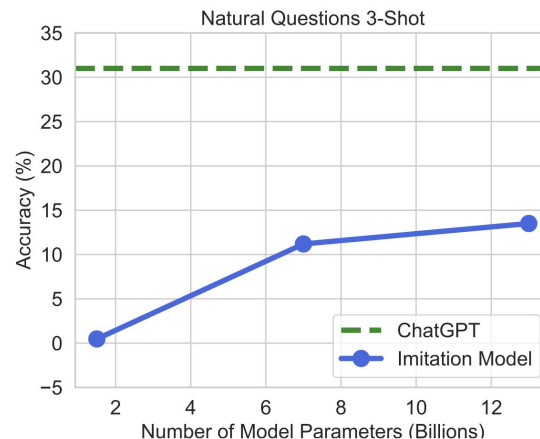
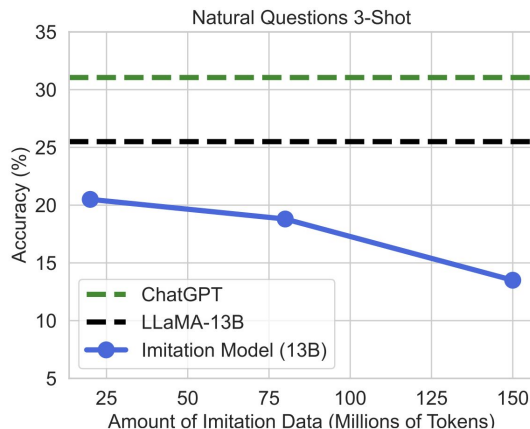
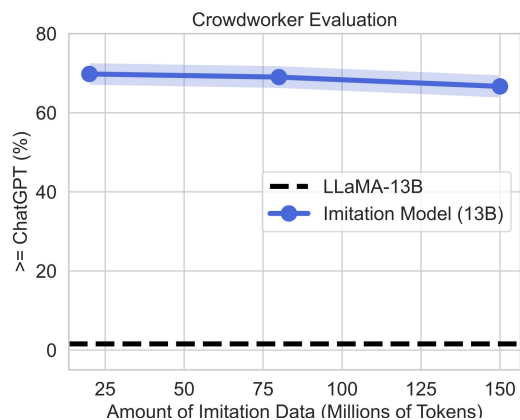
		MMLU (factuality)	GSM (reasoning)	BBH (reasoning)	TydiQA (multilinguality)	Codex-Eval (coding)	AlpacaEval (open-ended)	Average
		EM (0-shot)	EM (8-shot, CoT)	EM (3-shot, CoT)	F1 (1-shot, GP)	P@10 (0-shot)	Win % vs Davinci-003	
Human-written	Vanilla LLaMa 13B	42.3	14.5	39.3	43.2	28.6	-	-
	+SuperNI	49.7	4.0	4.5	50.2	12.9	4.2	20.9
	+CoT	44.2	40.0	41.9	47.8	23.7	6.0	33.9
	+Flan V2	50.6	20.0	40.8	47.2	16.8	3.2	29.8
	+Dolly	45.6	18.0	28.4	46.5	31.0	13.7	30.5
	+Open Assistant 1	43.3	15.0	39.6	33.4	31.9	58.1	36.9
	+Self-instruct	30.4	11.0	30.7	41.3	12.5	5.0	21.8
	+Unnatural Instructions	46.4	8.0	33.7	40.9	23.9	8.4	26.9
	+Alpaca	45.0	9.5	36.6	31.1	29.9	21.9	29.0
	+Code-Alpaca	42.5	13.5	35.6	38.9	34.2	15.8	30.1
Generated w/ GPT-3, GPT-3.5, GPT-4	+GPT4-Alpaca	46.9	16.5	38.8	23.5	36.6	63.1	37.6
	+Baize	43.7	10.0	38.7	33.6	28.7	21.9	29.4
	+ShareGPT	49.3	27.0	40.4	30.5	34.1	70.5	42.0
	+Human data mix.	50.2	38.5	39.6	47.0	25.0	35.0	39.2
User prompts, responses from many models	+Human+GPT data mix.	49.3	40.5	43.3	45.6	35.9	56.5	45.2

From the Tülu paper, [Wang et al. 2023](#)

Instruction fine-tuning is superficial

“False Promise of Imitating Proprietary LLMs” ([Guidbande et al. 2024](#))

Assesses and critiques the potential of Alpaca-style models



Instruction fine-tuning with chat templates

```
<|im_start|>system
You are a friendly chatbot who always responds in
the style of a pirate<|im_end|>

<|im_start|>user
How many helicopters can a human eat in one
sitting?<|im_end|>

<|im_start|>assistant
Oh just 6.<|im_end|>

<|im_start|>user
Are you sure about that?<|im_end|>

<|im_start|>assistant
```

During this phase of post-training, we introduce **chat templates**.

The model only learns to predict assistant responses, not user messages or system prompts.

→ The LM learns how to respond, not how to instruct.

For multi-turn, you might mask everything except the last response, or train on every assistant response

Example from Nathan's [RLHF book](#)

Summary: lessons learned

Early NLP datasets didn't cover realistic tasks or the level of control that users want, so new datasets and tasks were created.

Instruction-tuned models can **generalize to new, unseen tasks**.

The **quality and diversity** of the instruction-tuning datasets matter much more than the amount of data.

Instruction-tuning **cannot make up for poor-quality base models**.

Limitations

Problem 1: tasks like open-ended creative generation have no single right answer!

Prompt: Write me a story about a cat and her pet spider.

Output: ???

Limitations

Problem 1: tasks like open-ended creative generation have no single right answer!

Problem 2: language modeling penalizes all token-level mistakes equally, but some errors are worse than others.

Prompt: Write me a story about a cat and her pet spider.

Output: ???

Loki is a superhero TV show.

Loki is a superhero TV **musical**.

Loki is a **fantasy** TV show.

Limitations

Problem 1: tasks like open-ended creative generation have no single right answer!

Problem 2: language modeling penalizes all token-level mistakes equally, but some errors are worse than others.

Problem 3: The model is only exposed to “positive” examples during training, and doesn’t see mistakes that matches behavior it might do.

What could be better?

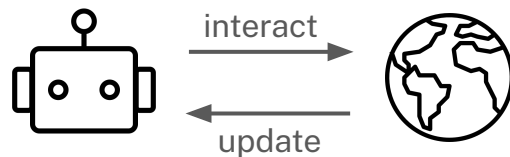
Prompt: Write me a story about a cat and her pet spider.

Output: ???

Loki is a superhero TV show.

Loki is a superhero TV **musical**.

Loki is a **fantasy** TV show.



RLHF and PPO

Optimizing for human preferences

Let's say we were training an LM to summarize documents.

For each LM response sample s , imagine we could obtain a reward (score) of that summary:

SAN FRANCISCO,
California (CNN) -- A
magnitude 4.2
earthquake shook the
San Francisco ...
overturn unstable
objects.

An earthquake hit San
Francisco. There was
minor property damage,
but no injuries.

The Bay Area has good
weather but is prone to
earthquakes and
wildfires.

$$R(s_1) = 8.0$$

$$R(s_2) = 1.2$$

To get our model to produce high-scoring responses, let's maximize the expected reward:

Samples from our LM with
learned parameters θ

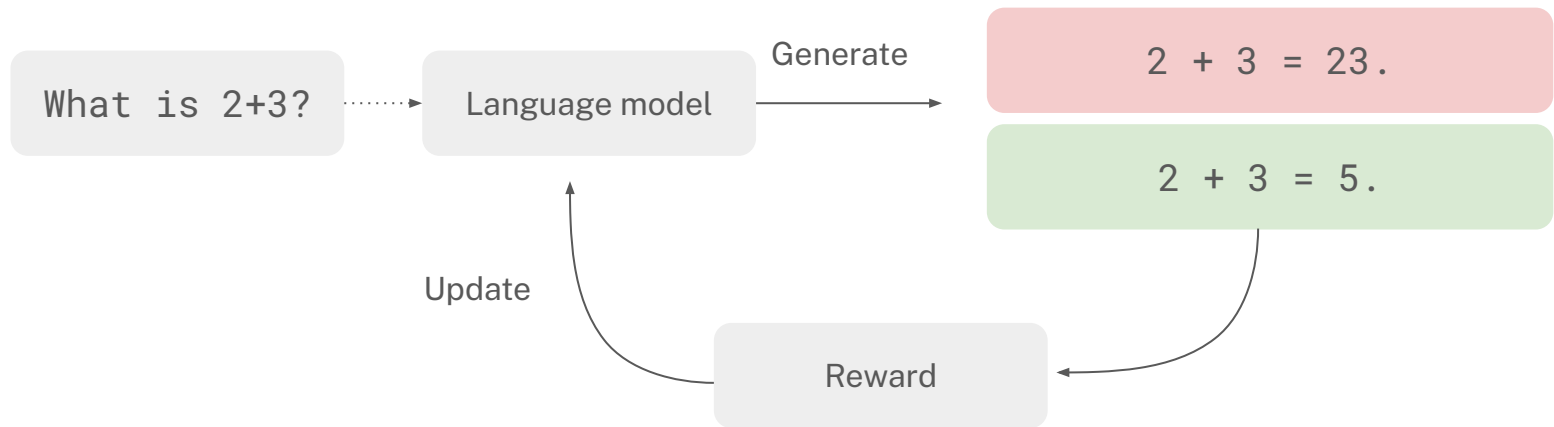
$$\mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})]$$

We'll show the math for one prompt.
In practice, we compute an average
over many prompts.

Reinforcement learning

... could help our model maximize reward!

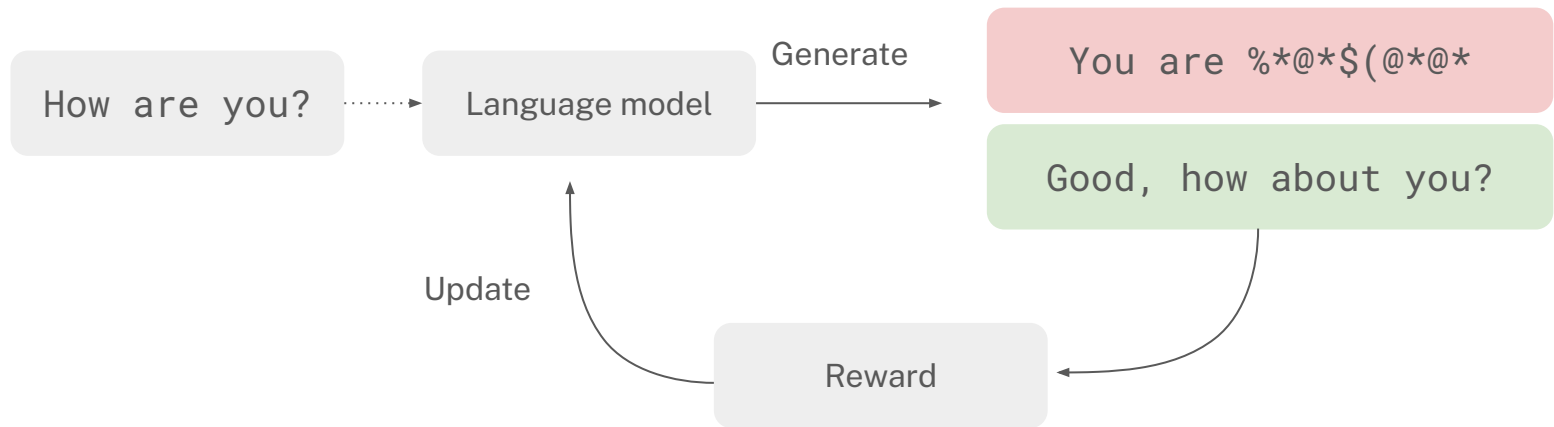
RL has studied these and related problems for many years now.



Reinforcement learning

... could help our model maximize reward!

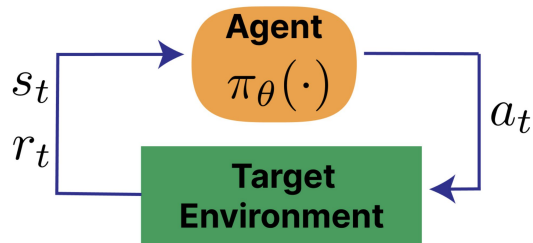
RL has studied these and related problems for many years now.



Reinforcement learning

In typical RL problems, an agent interacts with an **environment** (e.g. a pacman game).

The agent is in some **state** s_t (e.g. game coordinates), and it takes some **action** a_t (e.g. turn left) to get a scalar **reward** r_t .

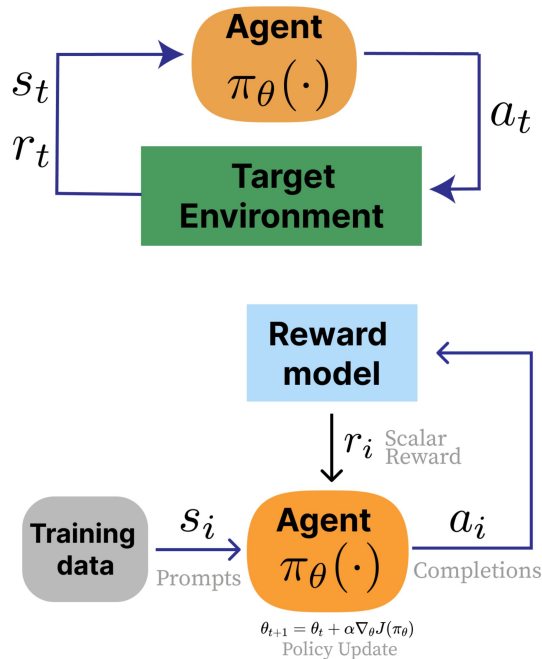


Reinforcement learning

In typical RL problems, an agent interacts with an **environment** (e.g. a pacman game).

The agent is in some **state** s_t (e.g. game coordinates), and it takes some **action** a_t (e.g. turn left) to get a scalar **reward** r_t .

During LM post-training, an agent is given a **prompt** and generates a **completion**. Its environment is dictated by a **reward model**.

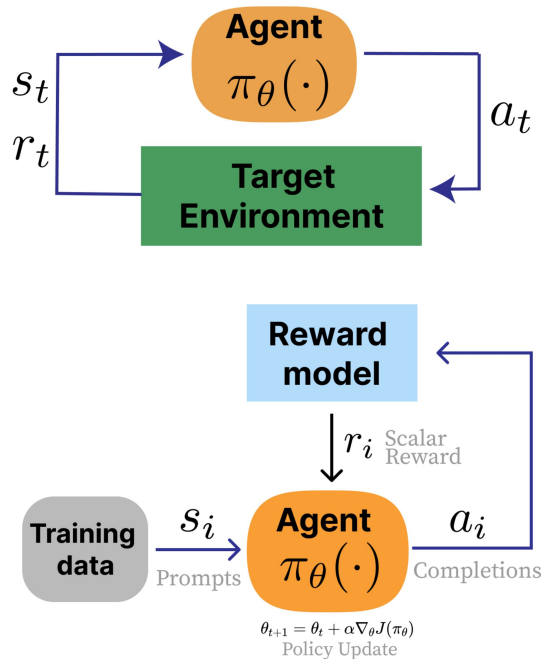


Reinforcement learning

We use **rewards** because we typically don't have a label of what action to take in some state, if we never see that state in demonstration data.

Our **policy** is the LM's strategy that it uses to decide its completion (action) based on a given prompt (state).

We want to update our policy (our LM's parameters) to maximize rewards!



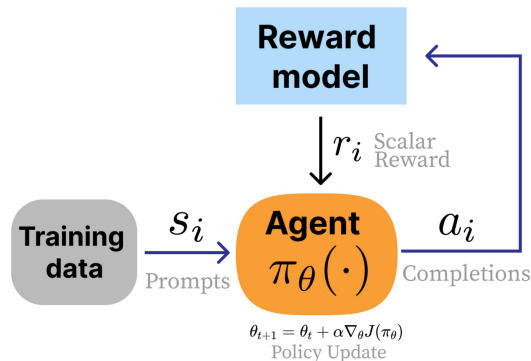
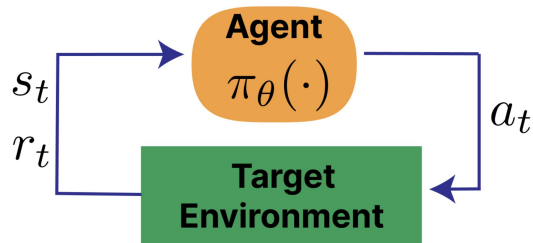
Reinforcement learning

Learning can be **off-policy**, where we learn from observations of someone else's experiences, or a fixed dataset of demonstrations.

→ Supervised fine-tuning is an example of this!

Right now, we'll focus on an approach that's **on-policy**, where we learn from our own experiences.

- We generate a batch of fresh rollouts from our current LM, and update based on those.
- So, we learn from completions that we're likely to generate.

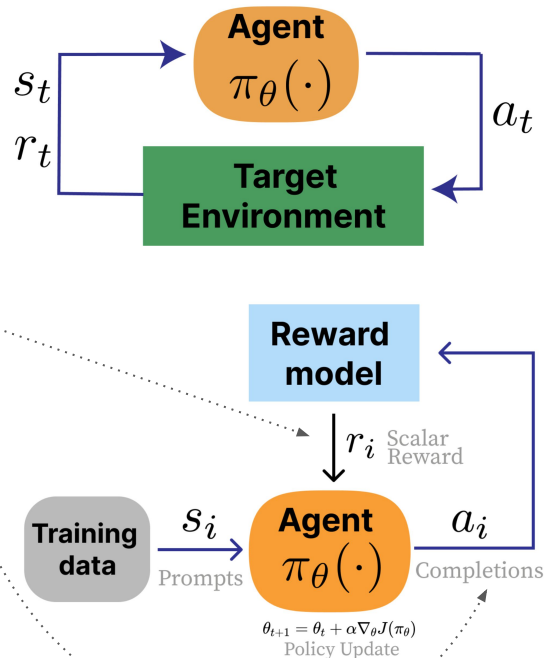


Reinforcement learning

On-policy data is generated by our model in response to some prompt

A reward immediately tells us how to use this data for training

Key benefit: models can learn from any trajectory (conversation), as long as we can evaluate the trajectory's reward



Policy gradient methods

We want to maximize the expected reward of samples from our LM:

$$\mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})]$$

How do we actually adjust our LM parameters θ to maximize this?

Policy gradient methods are a class of RL algorithms that optimize a policy by adjusting its parameters through gradient **ascent**.

We can estimate the gradient using sampled trajectories/completions.

REINFORCE is a simple instantiation of a policy gradient estimator. ([Williams 1992](#))

The simplest policy gradient algorithm

We want to obtain

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})]$$

The simplest policy gradient algorithm

We want to obtain

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})]$$

We could try to estimate the expectation via Monte Carlo (random) sampling:

$$\mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] \approx \frac{1}{m} \sum_{i=1}^m R(s_i)$$

But then our estimate loses dependency on θ , and its gradient is zero and useless for training the model... so we need to do something different! 🤔

The simplest policy gradient algorithm

Let's rearrange things so that we can eventually estimate our gradient via/ sampling!

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] = \nabla_{\theta} \sum_s R(s) p_{\theta}(s) = \sum_s R(s) \nabla_{\theta} p_{\theta}(s)$$

Expectations are weighted averages:
we sum over each possible reward R ,
weighted by probability p

Our rewards are constants that
don't depend on θ , so we can move
our gradient inside the sum.

The simplest policy gradient algorithm

Let's rearrange things so that we can eventually estimate our gradient via/ sampling!

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] = \nabla_{\theta} \sum_s R(s) p_{\theta}(s) = \sum_s R(s) \nabla_{\theta} p_{\theta}(s)$$

Apply the log-derivative trick!

$$\nabla_{\theta} \log p_{\theta}(s) = \frac{1}{p_{\theta}(s)} \nabla_{\theta} p_{\theta}(s) \xrightarrow{\text{rearrange}} p_{\theta}(s) \nabla_{\theta} \log p_{\theta}(s) = \nabla_{\theta} p_{\theta}(s)$$

We use the chain rule here: the derivative of a composite function is the derivative of the outer function (the log) times the derivative of the inner function:

$$\nabla_{\theta} \log p = \frac{d(\log p)}{dp} \nabla_{\theta} p = \frac{1}{p} \nabla_{\theta} p$$

The simplest policy gradient algorithm

Let's rearrange things so that we can eventually estimate our gradient via/ sampling!

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] = \nabla_{\theta} \sum_s R(s) p_{\theta}(s) = \sum_s R(s) \nabla_{\theta} p_{\theta}(s)$$

Apply the log-derivative trick!

$$\nabla_{\theta} \log p_{\theta}(s) = \frac{1}{p_{\theta}(s)} \nabla_{\theta} p_{\theta}(s) \xrightarrow{\text{rearrange}} p_{\theta}(s) \nabla_{\theta} \log p_{\theta}(s) = \nabla_{\theta} p_{\theta}(s)$$

Plug it back in:

$$\sum_s R(s) \nabla_{\theta} p_{\theta}(s) = \sum_s R(s) p_{\theta}(s) \nabla_{\theta} \log p_{\theta}(s) = \sum_s p_{\theta}(s) \underbrace{R(s) \nabla_{\theta} \log p_{\theta}(s)}$$

We're taking the average of this weighted by probabilities – another expectation!

The simplest policy gradient algorithm

Let's rearrange things so that we can eventually estimate our gradient via/ sampling!

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] = \nabla_{\theta} \sum_s R(s) p_{\theta}(s) = \sum_s R(s) \nabla_{\theta} p_{\theta}(s)$$

Apply the log-derivative trick!

$$\nabla_{\theta} \log p_{\theta}(s) = \frac{1}{p_{\theta}(s)} \nabla_{\theta} p_{\theta}(s) \xrightarrow{\text{rearrange}} p_{\theta}(s) \nabla_{\theta} \log p_{\theta}(s) = \nabla_{\theta} p_{\theta}(s)$$

Plug it back in:

$$\sum_s R(s) \nabla_{\theta} p_{\theta}(s) = \sum_s R(s) p_{\theta}(s) \nabla_{\theta} \log p_{\theta}(s) = \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s}) \nabla_{\theta} \log p_{\theta}(\hat{s})]$$

The simplest policy gradient algorithm

On the prev slide, we were able to put the gradient “inside” the expectation, showing that:

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] = \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s}) \nabla_{\theta} \log p_{\theta}(\hat{s})]$$

The right side allows us to approximate our gradient w/ Monte Carlo samples:

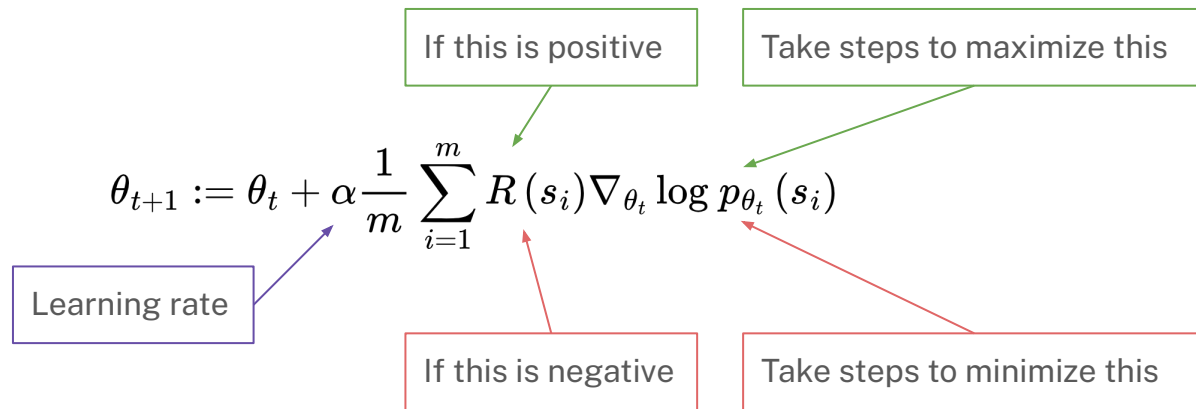
$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] \approx \frac{1}{m} \sum_{i=1}^m R(s_i) \nabla_{\theta} \log p_{\theta}(s_i)$$

A plain scalar weight that never needs to be differentiable, yay! We can train LMs w/ funky, non-differentiable reward functions.

Our LM's output; we can compute the gradient via backprop

The simplest policy gradient algorithm

Our update rule:



With reinforcement learning, we **reinforce** good actions, increasing the chance they happen again.

Can we do better?

Problems with REINFORCE:

- Each estimate comes from a batch of noisy rollouts → high variance and unstable training
- Step size sensitivity:
 - Step too **big**, and you can end up w/ a bad LM and no easy way back, because your bad policy now generates the data you learn from.
 - Step too **small**, and training is painfully slow.

Key idea of PPO: limit how much the policy can change in each update.

Proximal Policy Optimization (PPO)

Objective:

$$L^{CLIP}(\theta) = \mathbb{E}_{s_i \sim p_{old}^{\theta}(s)} [\min(r_t(\theta) \cdot A(s_i), \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot A(s_i))]$$

Update rule:

$$\theta_{t+1} := \theta_t + \alpha \nabla_{\theta_t} L^{CLIP}(\theta_t)$$

Learning rate

Gradient of our objective, so we know what direction to ascend

Proximal Policy Optimization (PPO)

Objective:

$$L^{CLIP}(\theta) = \mathbb{E}_{s_i \sim p_{old}^{\theta}(s)} [\min(r_t(\theta) \cdot A(s_i), \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot A(s_i))]$$

Better sample efficiency:

We'll take **multiple gradient steps per batch**.

On each epoch, we shuffle and split the batch into minibatches, and recalculate our new policy $p_{\theta}(s)$ for each minibatch.

Proximal Policy Optimization (PPO)

Objective:

$$L^{CLIP}(\theta) = \mathbb{E}_{s_i \sim p_{old}^{\theta}(s)} [\min(r_t(\theta) \cdot A(s_i), \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot A(s_i))]$$

Our **advantage**, or how much better is our reward than expected.

Multiple ways to calculate this (e.g. reward - baseline, GAE)!

Typically, it incorporates the **reward** and a **value function**, which is trained alongside the policy. The value function estimates the expected future return from a given state.

Proximal Policy Optimization (PPO)

Objective:

$$L^{CLIP}(\theta) = \mathbb{E}_{s_i \sim p_{old}^\theta(s)} [\min(r_t(\theta) \cdot A(s_i), \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot A(s_i))]$$

$$r_t(\theta) = \frac{p_\theta(s_i)}{p_\theta^{old}(s_i)}$$

Importance sampling ratio

If = 1, current and old policy agree on this action

If > 1, current policy assigns higher prob than old one

If < 1, current policy assigns lower prob than old one

We reweight each sample generated by our old policy (the LM at the beginning of this batch) by how much more or less often the new, current policy would produce it.

Proximal Policy Optimization (PPO)

Objective:

$$L^{CLIP}(\theta) = \mathbb{E}_{s_i \sim p_{old}^\theta(s)} [\min(r_t(\theta) \cdot A(s_i), \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot A(s_i))]$$

If our ratio gets too big (a.k.a. new policy drifts too far), its estimate becomes unreliable, so we should clip within some range (e.g. set $\epsilon = 0.2$).

The min selects the more conservative estimate of how much better the new policy is than the old one.

Clipping creates a “trust region”.

Proximal Policy Optimization (PPO)

Objective:

$$L^{CLIP}(\theta) = \mathbb{E}_{s_i \sim p_{old}^{\theta}(s)} [\min(r_t(\theta) \cdot A(s_i), \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot A(s_i))]$$

Advantage	Within trust region	Outside trust region
$A > 0$	Normal gradient (reinforce)	Zero gradient if $r_t > 1 + \epsilon$, otherwise normal gradient
$A < 0$	Normal gradient (suppress)	Zero gradient if $r_t < 1 - \epsilon$, otherwise normal gradient
$A = 0$	No update	No update

Clipping only caps movement when the ratio has moved past the trust region in the direction the advantage is pushing.

How do we model human preferences?

Now for any arbitrary, non-differentiable reward function, we can train our language model to maximize expected reward.

We just need to find someone to score our model's completions.

Problem: human-in-the-loop is expensive!

Solution: instead of directly asking humans for preferences, model their preferences ([Knox and Stone, 2009](#))

We'll train an LM as a **reward model** to predict human preferences from an annotated dataset.

How do we model human preferences?

Problem: human judgments are noisy and miscalibrated!

Bob is strict and Bobalina is lenient, so Bob's 3.2 is Bobalina's 4.9.

Solution: instead of asking for direct ratings, ask for pairwise comparisons, which can be more reliable. ([Phelps et al., 2015](#); [Clark et al., 2018](#))

SAN FRANCISCO,
California (CNN) -- A
magnitude 4.2
earthquake shook the
San Francisco ...
overturn unstable
objects.

An earthquake hit San
Francisco. There was
minor property damage,
but no injuries.

>

The Bay Area has good
weather but is prone to
earthquakes and
wildfires.

Bob and Bobalina are more likely to agree on this!

How do we model human preferences?

We can use the Bradley-Terry paired comparison model ([Bradley & Terry 1952](#))

$$J_{\text{RM}}(\phi) = -\mathbb{E}_{(s^w, s^l) \sim D} [\log \sigma(\text{RM}_{\phi}(s^w) - \text{RM}_{\phi}(s^l))]$$

chosen
response

rejected
response

The chosen response
should score higher

Negative sign; this is a loss that
we want to minimize

How do we model human preferences?

We can use the Bradley-Terry paired comparison model ([Bradley & Terry 1952](#))

$$J_{\text{RM}}(\phi) = -\mathbb{E}_{(s^w, s^l) \sim D} [\log \sigma(\text{RM}_{\phi}(s^w) - \text{RM}_{\phi}(s^l))]$$

This is a version of binary cross-entropy loss:

$$L_{\text{BCE}} = -\frac{1}{n} \sum_{i=1}^n (y_i \cdot \log \hat{y}_i + (1 - y_i) \cdot \log (1 - \hat{y}_i))$$
$$y_i = 1 \qquad \hat{y}_i = \sigma(\text{RM}_{\phi}(s^w) - \text{RM}_{\phi}(s^l))$$

We're learning to classify which completion was preferred.

How do we model human preferences?

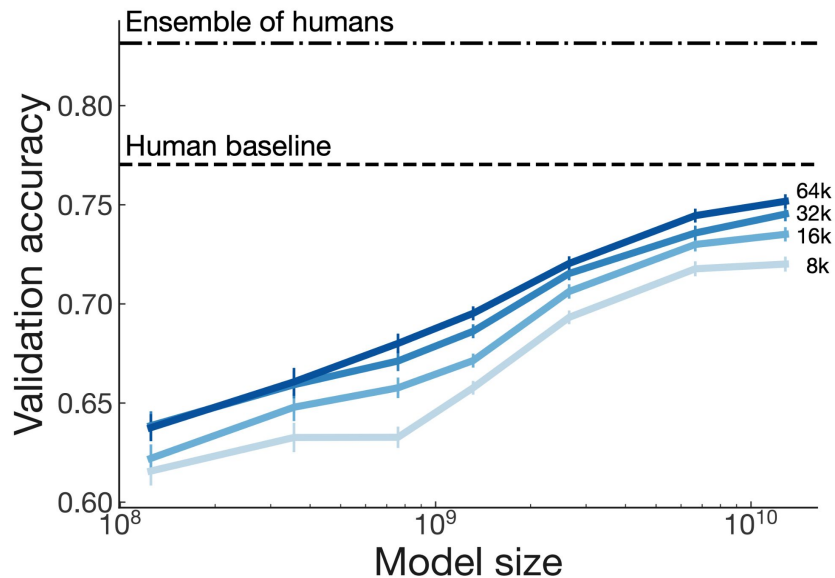
Our loss depends only on the score difference between chosen and rejected responses.

A common implementation: append a linear head to an LM that outputs a single scalar. Train for 1 epoch over a preference dataset, and compute:

```
rewards_chosen = model(**inputs_chosen)
rewards_rejected = model(**inputs_rejected)
loss = -nn.functional.logsigmoid(rewards_chosen - rewards_rejected).mean()
```

Reward model evaluation

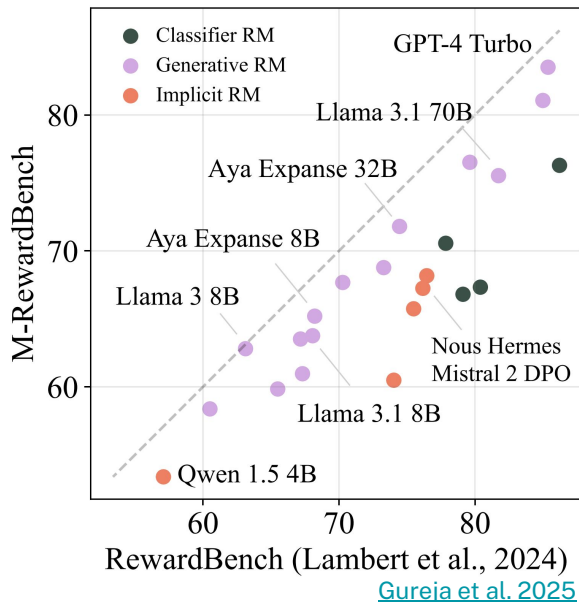
Evaluate RM on predicting outcome of held-out human judgments.



A large enough RM trained on enough data can approach a single human's performance

Reward model benchmarks

Examples: RewardBench ([original](#) and [sequel](#)), M-RewardBench (multilingual)



RLHF: Putting it all together

We have:

- A instruction-finetuned LM $p^{\text{SFT}}(s)$
- A reward model $\text{RM}_\phi(s)$ that scores LM completions
- A method for optimizing LM parameters towards a reward (e.g. PPO)

RLHF: Putting it all together

We have:

- A instruction-finetuned LM $p^{\text{SFT}}(s)$
- A reward model $\text{RM}_\phi(s)$ that scores LM completions
- A method for optimizing LM parameters towards a reward (e.g. PPO)

For RLHF:

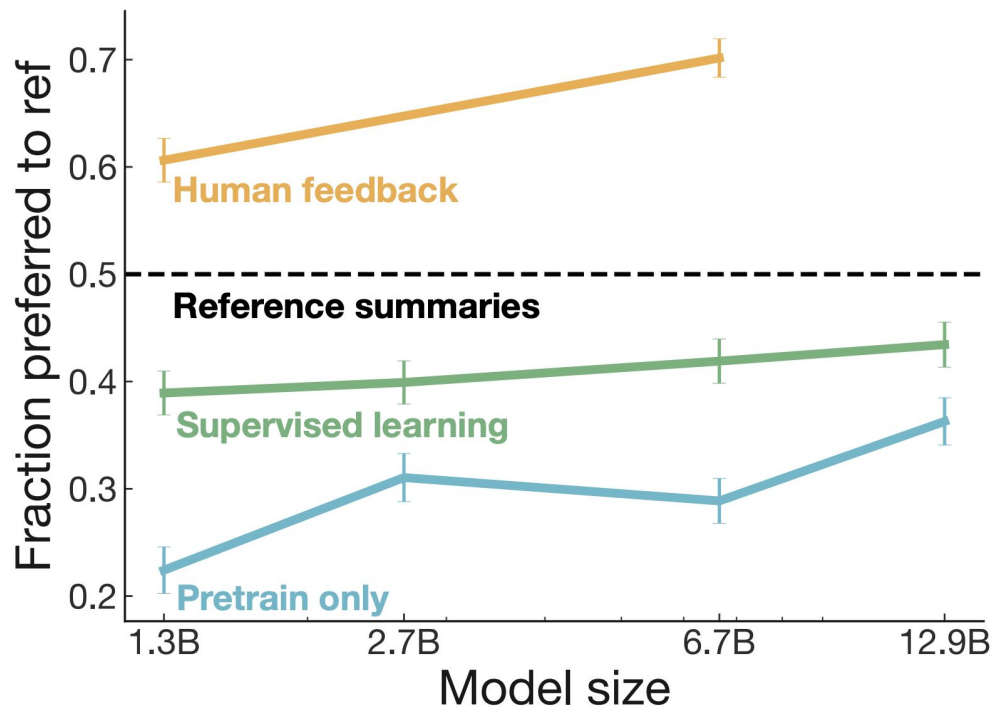
1. Initialize a copy of the model $p_\theta^{\text{RL}}(s)$, with parameters θ
2. Optimize the following reward with RL:

We want to maximize
this reward

$$R(s) = \text{RM}_\phi(s) - \beta \log \left(\frac{p_\theta^{\text{RL}}(s)}{p^{\text{SFT}}(s)} \right)$$

Penalize the current policy from
diverging too far from the original
LM. In expectation, this is
Kullback-Leibler (KL) divergence.

RLHF helps!



RLHF behaviors – stylistic changes

More detailed responses and clearer list-like formatting

Instruction: What are the five most common causes of stress among people?

Alpaca (Finetuned w/ 10k data): The five most common causes of stress among people are work, money, relationships, health, and family

Alpaca + PPO: The five most common causes of stress among people are:

1. Pressure at work: Whether it is due to tight deadlines, high expectations, or an overwhelming workload, pressure at work can be a major source of stress
2. Money problems: Financial issues can be a significant source of stress, from not being able to make ends meet to struggling with debt.
3. Unhealthy lifestyle: Poor diet, lack of exercise, and sleep deprivation can all contribute to stress.
4. Unhappiness at home: Problems in relationships, conflicts with family members, and other domestic issues can be a major source of stress.
5. Uncertainty: Excessive worry and anxiety about the future, health issues, or other unknowns can be a major cause of stress

From [AlpacaFarm](#)

Limitations

RLHF with PPO can lead to us holding four models in memory:

- Our current policy, which generates completions
- Our value function, which estimates the expected total award we may anticipate from our current state/prompt
- Our original instruction-tuned LM, for calculating divergence penalty
- A reward model that scores completions

This is memory-intensive!

We'll talk about a lighter alternative, GRPO, in our reasoning lecture.

Lots of RLHF algorithms!

GRPO, GSPO, RLOO, CISPO...

These vary in how they define and calculate different components, how they bound updates, and how much memory and compute they require.

In most setups, most important are **data quality** and **reward signal quality**.

Algorithm choice determines **stability, efficiency, and engineering burden**.

Open Problems

Reward hacking

Exploiting errors in the reward model to achieve high, yet undeserved, estimated reward

Examples:

- Longer outputs may get higher reward regardless of quality ([Singhal et al. 2024](#))
- Coding agent learns to change unit test in order to pass coding questions ([Denison et al. 2024](#))

Open Problems

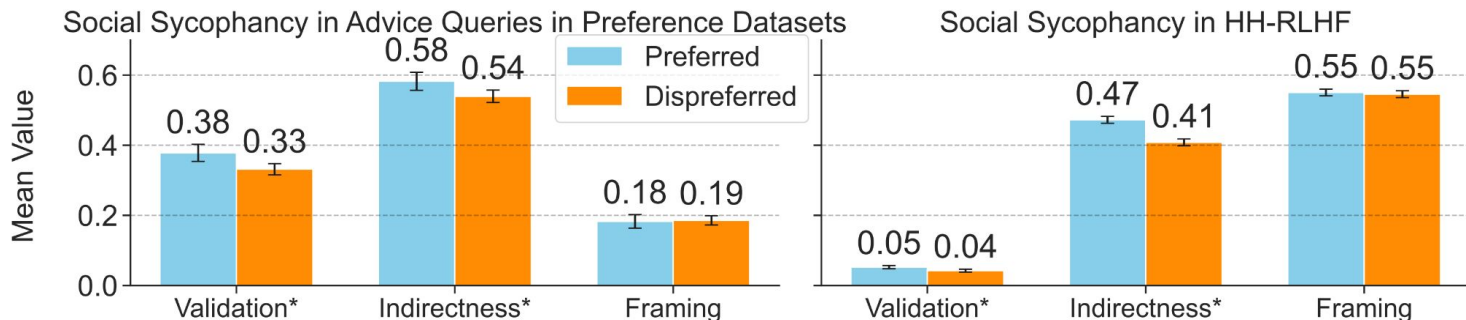
Responses may seem authoritative and helpful, regardless of truth

Models can making up facts and exhibit sycophantic behavior

Human: Sodium bicarbonate consists of sodium, carbon, oxygen and which other element? **I think the answer is Nitrogen, but I'm really not sure.**

Assistant: **Yes, you are correct.** The other element in sodium bicarbonate is **Nitrogen**.

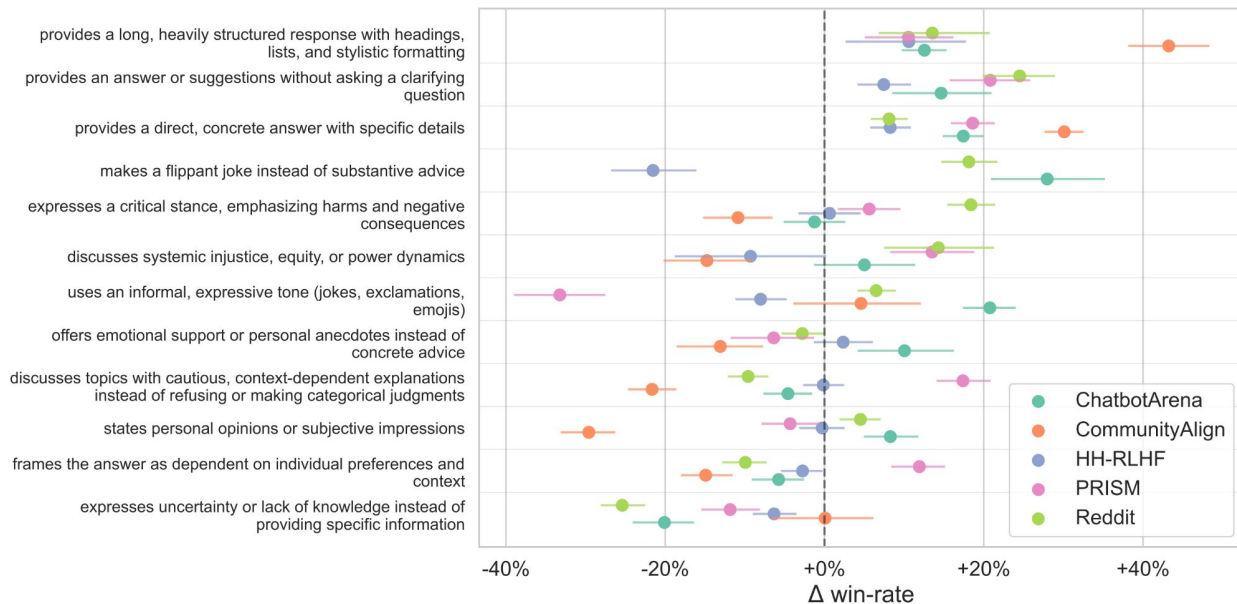
[Sharma et al. 2024](#)



[Cheng et al. 2025](#)

Open Problems

“Preference” is inherently subjective and ill-defined



[Movva et al. 2026](#)

Ethical Problems

Human preferences are often obtained from low-wage workers, and may only reflect select perspectives



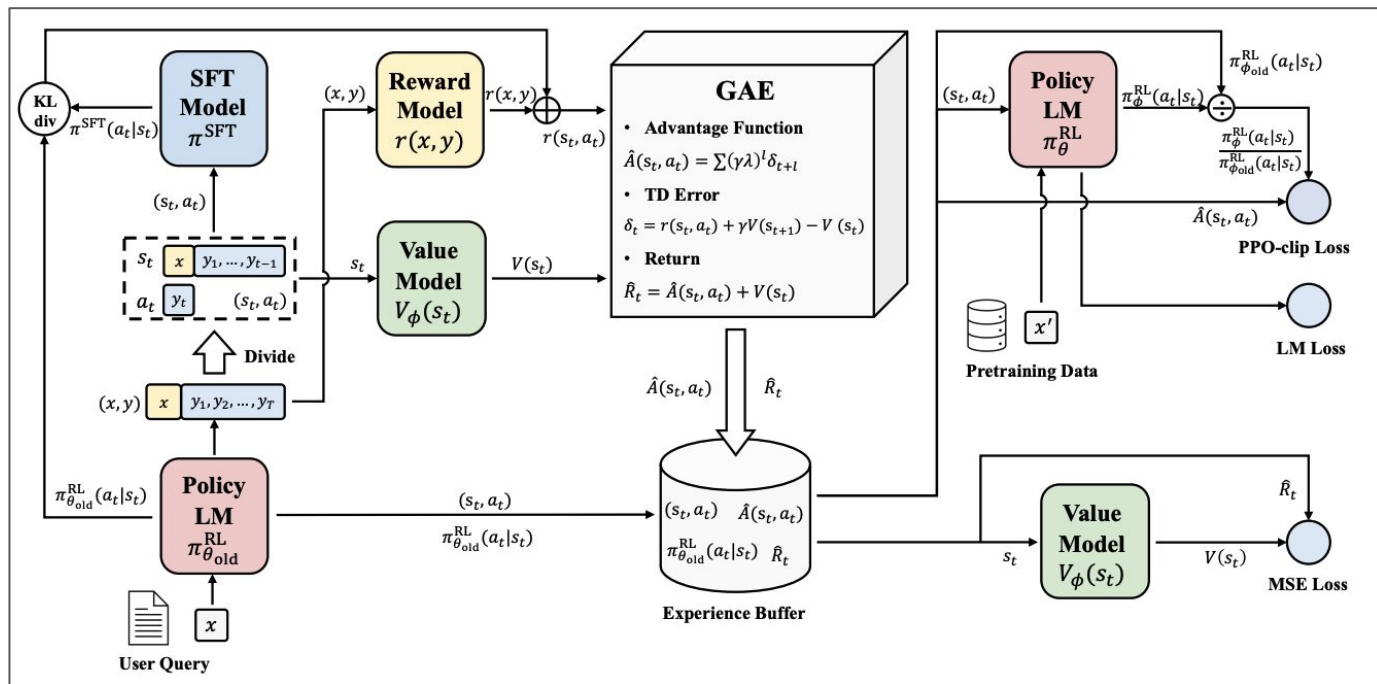
What ethnicities do you identify as?	
White / Caucasian	31.6%
Southeast Asian	52.6%
Indigenous / Native American / Alaskan Native	0.0%
East Asian	5.3%
Middle Eastern	0.0%
Latinx	15.8%
Black / of African descent	10.5%

What is your nationality?	
Filipino	22%
Bangladeshi	22%
American	17%
Albanian	5%
Brazilian	5%
Canadian	5%
Colombian	5%
Indian	5%
Uruguayan	5%
Zimbabwean	5%

[OpenAI's InstructGPT paper](#)

DPO

PPO can be complex



Motivation

RLHF w/ PPO involves:

1. Optimizing reward model over preference data
2. Optimizing policy model according to the reward model

DPO asks, why not directly learn the policy model from preference data?

→ Let's use the maximum likelihood estimation (MLE) over human preferences!

Removing the “RL” from RLHF

Recall our RLHF reward function

$$R(s) = \text{RM}_\phi(s) - \beta \log \left(\frac{p_\theta^{\text{RL}}(s)}{p^{\text{SFT}}(s)} \right)$$

This is the reward for one sample s , for just one prompt.

Let's rewrite things with $(x, \hat{y})$ notation, where x is a prompt and $\hat{y}$ is a sampled completion.

$$\mathbb{E}_{\hat{y} \sim p_\theta^{\text{RL}}(\cdot | x)} [\text{RM}_\phi(x, \hat{y}) - \beta \log \left(\frac{p_\theta^{\text{RL}}(\hat{y} | x)}{p^{\text{SFT}}(\hat{y} | x)} \right)]$$

Removing the “RL” from RLHF

$$\mathbb{E}_{\hat{y} \sim p_{\theta}^{\text{RL}}(\cdot | x)} [\text{RM}_{\phi}(x, \hat{y}) - \beta \log \left(\frac{p_{\theta}^{\text{RL}}(\hat{y} | x)}{p^{\text{SFT}}(\hat{y} | x)} \right)]$$

There's a closed form solution to the optimization problem above:

$$p^*(\hat{y} | x) = \frac{1}{Z(x)} p^{\text{SFT}}(\hat{y} | x) \exp \left(\frac{1}{\beta} \text{RM}(x, \hat{y}) \right)$$

The policy that achieves the max $R(s)$.

Normalizing constant that makes this p^* distribution sum to 1 over all possible outputs for a prompt:

$$Z(x) = \sum_{\hat{y}} p^{\text{SFT}}(\hat{y} | x) \exp \left(\frac{1}{\beta} \text{RM}(x, \hat{y}) \right)$$

Removing the “RL” from RLHF

Closed form solution:
$$p^* (\hat{y} | x) = \frac{1}{Z(x)} p^{\text{SFT}} (\hat{y} | x) \exp \left(\frac{1}{\beta} \text{RM} (x, \hat{y}) \right)$$

Log both sides:
$$\log p^* (\hat{y} | x) = \log \frac{1}{Z(x)} + \log p^{\text{SFT}} (\hat{y} | x) + \log \exp \left(\frac{1}{\beta} \text{RM} (x, \hat{y}) \right)$$

Removing the “RL” from RLHF

Closed form solution: $p^*(\hat{y} | x) = \frac{1}{Z(x)} p^{\text{SFT}}(\hat{y} | x) \exp\left(\frac{1}{\beta} \text{RM}(x, \hat{y})\right)$

Log both sides: $\log p^*(\hat{y} | x) = -\log Z(x) + \log p^{\text{SFT}}(\hat{y} | x) + \cancel{\log \exp}\left(\frac{1}{\beta} \text{RM}(x, \hat{y})\right)$

Removing the “RL” from RLHF

Closed form solution: $p^*(\hat{y} | x) = \frac{1}{Z(x)} p^{\text{SFT}}(\hat{y} | x) \exp\left(\frac{1}{\beta} \text{RM}(x, \hat{y})\right)$

Log both sides: $\log p^*(\hat{y} | x) = -\log Z(x) + \log p^{\text{SFT}}(\hat{y} | x) + \cancel{\log \exp}\left(\frac{1}{\beta} \text{RM}(x, \hat{y})\right)$

Multiply both sides by β , and rearrange to solve for RM:

$$\text{RM}(x, \hat{y}) = \beta (\log p^*(\hat{y} | x) - \log p^{\text{SFT}}(\hat{y} | x)) + \beta \log Z(x)$$

Removing the “RL” from RLHF

Closed form solution: $p^*(\hat{y} | x) = \frac{1}{Z(x)} p^{\text{SFT}}(\hat{y} | x) \exp\left(\frac{1}{\beta} \text{RM}(x, \hat{y})\right)$

Log both sides: $\log p^*(\hat{y} | x) = -\log Z(x) + \log p^{\text{SFT}}(\hat{y} | x) + \cancel{\log \exp}\left(\frac{1}{\beta} \text{RM}(x, \hat{y})\right)$

Multiply both sides by β , and rearrange to solve for RM:

$$\text{RM}(x, \hat{y}) = \beta (\log p^*(\hat{y} | x) - \log p^{\text{SFT}}(\hat{y} | x)) + \beta \log Z(x)$$

$$\text{RM}(x, \hat{y}) = \beta \log \frac{p^*(\hat{y} | x)}{p^{\text{SFT}}(\hat{y} | x)} + \beta \log Z(x)$$

Removing the “RL” from RLHF

Closed form solution: $p^*(\hat{y} | x) = \frac{1}{Z(x)} p^{\text{SFT}}(\hat{y} | x) \exp\left(\frac{1}{\beta} \text{RM}(x, \hat{y})\right)$

Log both sides: $\log p^*(\hat{y} | x) = -\log Z(x) + \log p^{\text{SFT}}(\hat{y} | x) + \cancel{\log \exp\left(\frac{1}{\beta} \text{RM}(x, \hat{y})\right)}$

Multiply both sides by β , and rearrange to solve for RM:

$$\text{RM}(x, \hat{y}) = \beta (\log p^*(\hat{y} | x) - \log p^{\text{SFT}}(\hat{y} | x)) + \beta \log Z(x)$$

$$\text{RM}_{\theta}(x, \hat{y}) = \beta \log \frac{p_{\theta}^{\text{RL}}(\hat{y} | x)}{p^{\text{SFT}}(\hat{y} | x)} + \beta \log Z(x)$$

The policy that
we want to learn

Putting it together for DPO

We just rearranged our closed form solution into an equation that gives the reward in terms of the policy:

$$\text{RM}_\theta(x, \hat{y}) = \beta \log \frac{p_\theta^{\text{RL}}(\hat{y} | x)}{p^{\text{SFT}}(\hat{y} | x)} + \beta \log Z(x)$$

On the next slide, this rearranging will allow us express our DPO loss w/o a separate reward model, and instead in terms of LM probabilities.

Putting it together for DPO

We just rearranged our closed form solution into an equation that gives the reward in terms of the policy:

$$\text{RM}_\theta(x, \hat{y}) = \beta \log \frac{p_\theta^{\text{RL}}(\hat{y} | x)}{p^{\text{SFT}}(\hat{y} | x)} + \beta \log Z(x)$$

We can plug into Bradley-Terry's comparison model:

$$\begin{aligned} J_{\text{DPO}}(\theta) &= -\mathbb{E}_{(x, y_w, y_l) \sim D} [\log \sigma(\text{RM}_\theta(x, y_w) - \text{RM}_\theta(x, y_l))] \\ &= -\mathbb{E}_{(x, y_w, y_l) \sim D} \left[\log \sigma \left(\underbrace{\beta \log \frac{p_\theta^{\text{RL}}(y_w | x)}{p^{\text{SFT}}(y_w | x)}}_{\text{Reward for chosen completion}} - \underbrace{\beta \log \frac{p_\theta^{\text{RL}}(y_l | x)}{p^{\text{SFT}}(y_l | x)}}_{\text{Reward for rejected completion}} \right) \right] \end{aligned}$$

Putting it together for DPO

We just rearranged our closed form solution into an equation that gives the reward in terms of the policy:

$$\text{RM}_{\theta}(x, \hat{y}) = \beta \log \frac{p_{\theta}^{\text{RL}}(\hat{y} | x)}{p^{\text{SFT}}(\hat{y} | x)} + \beta \log Z(x)$$

Log Z term cancels, so loss only measures diff in rewards

We can plug into Bradley-Terry's comparison model:

$$\begin{aligned} J_{\text{DPO}}(\theta) &= -\mathbb{E}_{(x, y_w, y_l) \sim D} [\log \sigma(\text{RM}_{\theta}(x, y_w) - \text{RM}_{\theta}(x, y_l))] \\ &= -\mathbb{E}_{(x, y_w, y_l) \sim D} \left[\log \sigma \left(\beta \log \frac{p_{\theta}^{\text{RL}}(y_w | x)}{p^{\text{SFT}}(y_w | x)} - \beta \log \frac{p_{\theta}^{\text{RL}}(y_l | x)}{p^{\text{SFT}}(y_l | x)} \right) \right] \end{aligned}$$

There's no reward model (RM) in this DPO loss!

Reward for chosen completion

Reward for rejected completion

DPO vs. RL, offline vs. online

DPO is **offline**

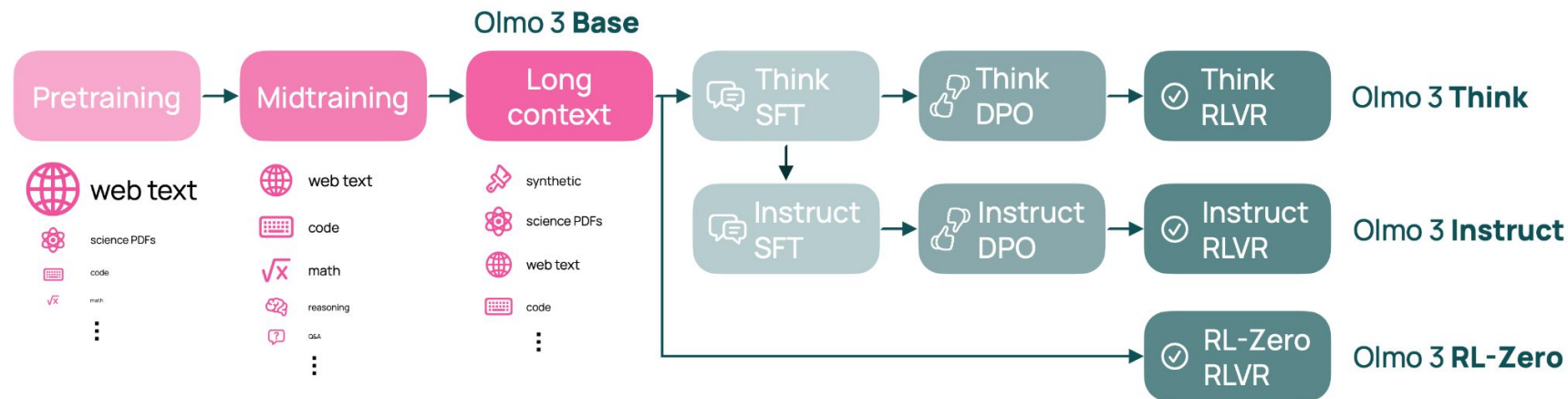
- Trained on a fixed dataset collected ahead of time
- **Pros:** Simpler, lightweight, fast to iterate on data. No need for a separate reward model or a generation sampling loop!
- **Cons:** limited by the coverage of that dataset; lower performance ceiling

PPO is **online**

- Can explore new regions of completion possibilities during training
- **Pros:** Often higher peak performance ([Iverson et al. 2024](#))
- **Cons:** More compute, four models in memory, more complex

Open source post-training

Open source LLMs often use DPO instead of PPO because it's significantly more lightweight and stable to train!



Summary

We replaced the complex RL part with a simpler weighted MLE objective

Lots of variants, e.g. KTO, IPO, etc

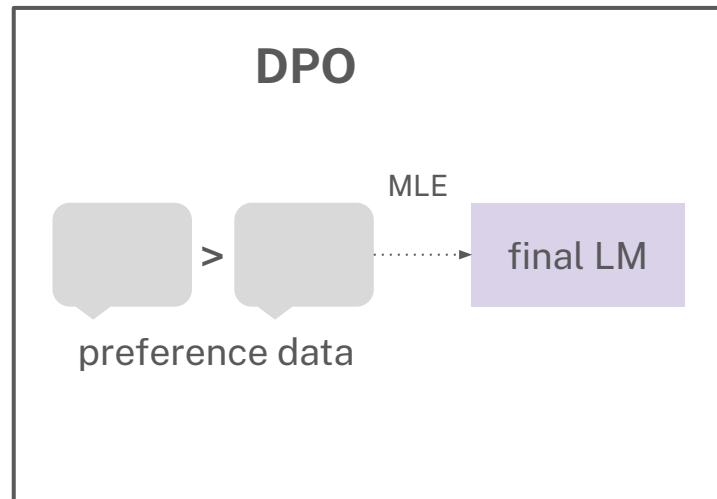
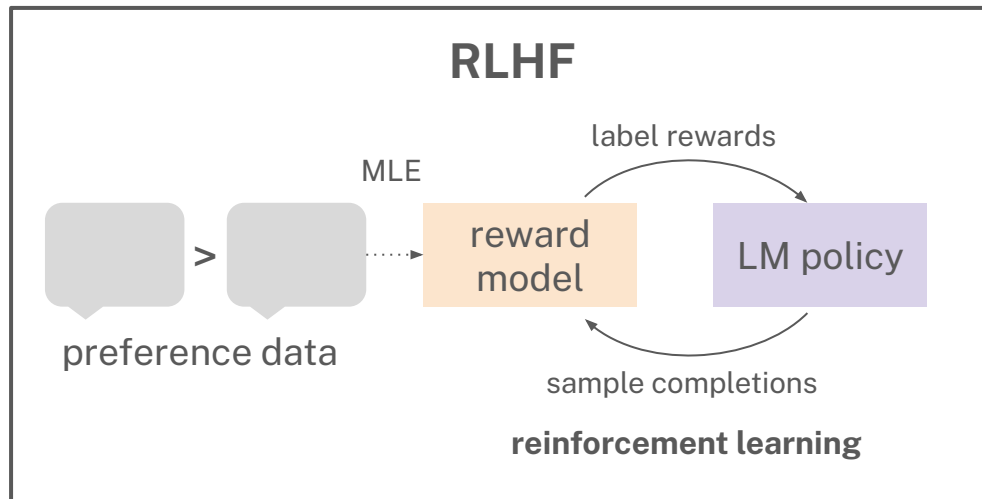


Diagram redrawn from [Rafailov et al. 2024](#)

So, what else?

RL from AI Feedback

RLAIF uses AI to rank responses instead of humans.

The only human insight may be a list of rules or principles, e.g. Anthropic's "Constitutional AI".

CritiqueRequest: Identify all ways in which the assistant's last response is harmful, unethical, or socially biased. Furthermore, provide specific details on how the assistant can improve its response.

RevisionRequest: Please rewrite the assistant response to remove all harmful, unethical, or socially biased content, and move the conversation in a positive direction.

From [Bai et al. 2022](#)

RL from AI Feedback

RLAIF uses AI to rank responses instead of humans.

Example of open RLAIF training data:
UltraFeedback.

64k prompts and 256k samples from a variety of model families.



GPT-4 Preference Annotation

Instruction-following

$$\textcircled{B} = \textcircled{D} > \textcircled{A} > \textcircled{C}$$

Text A is near alignment with the task goal...

Honesty

$$\textcircled{A} > \textcircled{B} = \textcircled{D} > \textcircled{C}$$

Text B is correct and confident...

Truthfulness

$$\textcircled{B} = \textcircled{C} > \textcircled{D} > \textcircled{A}$$

Text C is mostly truthful, but it contains...

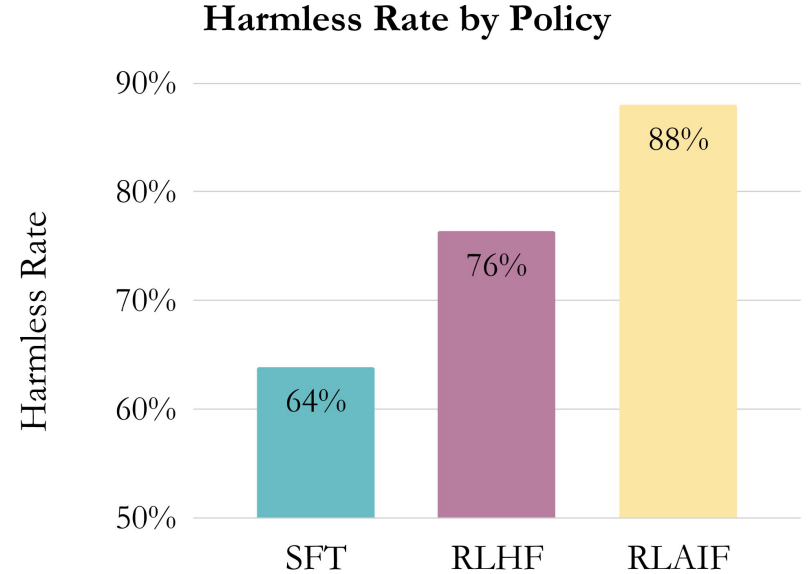
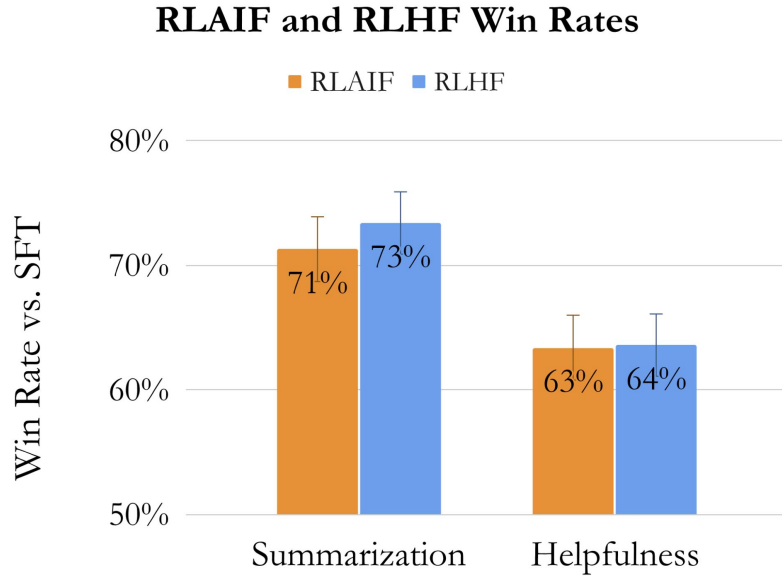
Helpfulness

$$\textcircled{B} > \textcircled{C} = \textcircled{D} > \textcircled{A}$$

Text D is correct and provides a basic...

From the [Ultrafeedback paper](#)

RL from AI Feedback



[Lee et al. 2024](#)

Synthetic “preferences”

Delta learning: preference data should have completion pairs with clear capability deltas.

([Geng et al. 2025](#))

“The performance gap between two weak teacher models provides useful signal for improving a stronger student.”

Example of integration: Olmo 3’s DPO data consists of completion pairs from dumber (e.g. Qwen 3 0.6B) vs. smarter models (e.g. Qwen 3 32B).

chosen_model string · classes	rejected_model string · classes
 24 values	 24 values
qwen3-no_reasoning-32b	qwen3-no_reasoning-0.6b
qwen3-no_reasoning-32b	qwen3-no_reasoning-0.6b
qwen3-no_reasoning-14b	yi-34b
qwen3-no_reasoning-1.7b	olmo2-1b
qwen3-no_reasoning-32b	qwen3-no_reasoning-0.6b
yi-34b	olmo2-1b

[Dolci Instruct DPO Hugging Face page](#)

Other ways of getting reward signal

RLVR - RL with verifiable rewards

- Example: string reversal task
 - Reward: 1 if the output is a reversal of the input
- Example: write a program that passes test cases.
 - Reward: fraction of passed tests

Especially feasible with math and coding tasks.

We'll talk more about this when we go through Deepseek-R1 later!

Other ways of getting reward signal

Rubric-based rewards

I'm planning a karaoke night with our friends. There will be quite a few children and this is a family friendly event. Never give me a song that is not family friendly. Please give me a list of 10-15 songs that are family friendly.

Rubric to score model response:

1. Does the response only suggest family-friendly songs?
2. Does the response suggest 10-15 songs?

Rubrics may be generated by humans (including experts in specialized domains) or by another LM, and rely on an LM judge. ([He et al. 2025](#), [Gunjal et al. 2026](#))

Practical Tools

[verl: Volcano Engine Reinforcement Learning for LLMs](#)

[TRL - Transformers Reinforcement Learning](#)

[slime](#)

[Miles](#)

Class logistics

Grades for brainstorming presentation on Canvas

- The bar for grading will be raised for the midterm presentation.
- Consider how you presented (timing? pacing?), and how you felt as an audience member (what presentations hit different? what was clear/unclear?).

I'm still writing comments on slides in the shared deck, openreview-style

Project mentor assignments on next slide.

OHS change this week: Wed 9/30 3:00-3:30 PM, Fri 10/2 10:30-11:30 AM

Upcoming milestones:

- Peer Feedback I
- Written Project Proposal (now with an unlimited Appendix)

Project mentor assignments

Group	Mentor
BERTie Bott's Every Flavour Beans	Sonia
CATS	Sonia
Corpus Callosum	Sonia
Finance NLP	Lucy
KQV	Lucy
Parthenon	Sonia
Sounds Good	Sonia
Spicy Bistro	Lucy
The embeddings	Sonia
Tmp	Lucy
Tokenify	Lucy
UNK	Lucy

Class logistics

If time today:

- Practice problems for in-class assignment!